Revision — 1.3.2 Databases

OCR H446 — one-page revision summary

Must know

- **Flat file** = single table, high redundancy, prone to anomalies. **Relational** = multiple linked tables, reduced redundancy, consistent, scalable.
- **Entity** = thing data is stored about (→ table); **attribute/field** = column; **record/tuple** = one row.
- **Keys**: **Primary** uniquely identifies each record; **Foreign** = the primary key of *another* table, used to link tables; **Composite** = a primary key made of 2+ attributes; **Secondary** = a non-primary attribute indexed for fast searching.
- **ER modelling**: relationships are 1:1, 1:M or M:N. A **many-to-many** is resolved with a **linking/junction table** that holds a composite key of the two foreign keys.
- **Referential integrity**: every foreign key must reference a valid existing primary key (or be null) → no orphaned records.
- **Normalisation** organises data to reduce redundancy and remove insertion/update/deletion anomalies.
- **1NF**: no repeating groups, atomic values, has a primary key. **2NF**: 1NF + no **partial** dependencies (every non-key attribute depends on the *whole* key). **3NF**: 2NF + no **transitive** dependencies (no non-key attribute depends on another non-key attribute).
- **Transaction** = one logical operation; all steps succeed or all fail. **ACID** = **A**tomicity, **C**onsistency, **I**solation, **D**urability.
- **Record locking** locks a record during a transaction to prevent the **lost update problem** from concurrent edits; risk of **deadlock**. Note two meanings of *redundancy*: data redundancy (bad, removed by normalisation) vs hardware/backup redundancy (good, for reliability).

Key terms

Term	Definition
Primary key	Attribute(s) that uniquely identify each record in a table
Foreign key	Primary key of another table, used to link tables
Composite key	Primary key made up of two or more attributes
Referential integrity	Foreign keys must reference a valid existing primary key (or be null)
Normalisation	Organising data to reduce redundancy and remove anomalies
Transitive dependency	A non-key attribute depending on another non-key attribute (removed in 3NF)
Transaction (ACID)	All-or-nothing operation obeying Atomicity, Consistency, Isolation, Durability
Record locking	Locking a record during a transaction to stop concurrent edits

Worked example — normalisation to 3NF

Unnormalised `Order(OrderID, CustName, Item1, Item2, ...)` : - **1NF** — remove the repeating group of items into their own rows: `OrderLine(OrderID, ItemID, Qty)` with atomic values. - **2NF** — an item's

`ItemName` depends only on `ItemID` (part of a composite key), a **partial dependency**. Move it out: `Item(ItemID, ItemName, Price)`. - **3NF** — in `Order(OrderID, CustID, CustName, CustAddress)`, `CustName` / `CustAddress` depend on `CustID`, not `OrderID` — a **transitive dependency**. Move them out: `Customer(CustID, CustName, CustAddress)`, leaving `Order(OrderID, CustID)`.

Example SQL join: list each member's name and the books they have on loan.

```
SELECT Member.Name, Loan.BookTitle
FROM Member
JOIN Loan ON Member.MemberID = Loan.MemberID;
```

Exam tips

- A foreign key is the **primary key of another table** — always name *which* table it links to.
- For each normalisation stage, name the dependency removed (repeating group → 1NF, partial → 2NF, transitive → 3NF) and say *why* (removes redundancy/anomalies): "the key, the whole key, and nothing but the key."
- In SQL keep the order `SELECT ... FROM ... WHERE ... ORDER BY`; put text values in quotes and use `ASC` / `DESC` to sort.
- For ACID, link each property to a real consequence (e.g. Atomicity → rollback on failure so no partial change).